

Contents

Lab6 — Contact Management API, Step by Step	1
Step 0 — Set up VS Code	1
Step 1 — Create the project	1
Step 2 — The Contact model	2
Step 3 — The in-memory collection, behind an interface	2
Step 4 — Wire it up with Dependency Injection	3
Step 5 — The REST controller	4
Step 6 — Run it and try every CRUD operation	5
Checklist against the exercise	6

Lab6 — Contact Management API, Step by Step

Goal (from `lab6_asp-getting-started.pdf`, “Backend Exercise” slide): a REST controller for a simple Contact-Management system, all CRUD operations, data stored in an internal in-memory collection.

Prerequisites: .NET 7 or 8 SDK installed, VS Code.

Step 0 — Set up VS Code

1. Install the **C# Dev Kit** extension (search for it in the Extensions panel, `Ctrl+Shift+X` / `Cmd+Shift+X`). It bundles IntelliSense, debugging, and project support. It pulls in the base “C#” extension automatically.
2. Optional but handy for Step 6: install the **REST Client** extension — lets you send HTTP requests straight from a `.http` file with a clickable “Send Request” link, no separate tool needed.
3. Confirm the SDK is visible to VS Code by opening the integrated terminal (`Ctrl+`` / `Cmd+``) and running:

```
dotnet --version
```

You should see `8.0.x` or similar. If it’s not found, install the .NET 8 SDK first and restart VS Code.

Step 1 — Create the project

Do this from the integrated terminal — no wizard needed:

```
dotnet new webapi -n ContactManagement
cd ContactManagement
code .
```

`code .` reopens VS Code with this folder as the workspace root (do this even if VS Code is already open — it keeps paths consistent for the debugger). The template scaffolds a controller-based Web API and also drops in a `WeatherForecast` sample. In the VS Code Explorer panel, delete the two sample files, you won’t need them:

- `WeatherForecast.cs`
- `Controllers/WeatherForecastController.cs`

The first time you open a `.cs` file, VS Code may prompt “Required assets to build and debug are missing. Add them?” — click **Yes**. This generates a `.vscode/launch.json` and `.vscode/tasks.json` for you, which is what makes `F5` work later.

Run it once to confirm the empty shell works:

```
dotnet run
```

You should see it start listening on something like `http://localhost:5xxx`. Stop it with `Ctrl+C` once confirmed.

Step 2 — The Contact model

The task list asks for “a model class for a Contact”. In the Explorer panel, right-click the project root → **New Folder** → **Models**, then right-click **Models** → **New File** → **Contact.cs**:

```
namespace ContactManagement.Models;

public class Contact
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;
}
```

Step 3 — The in-memory collection, behind an interface

The task list asks for “a collection of the Contacts as list or dictionary”. Rather than putting a `List<Contact>` straight into the controller, wrap it behind an interface. This costs nothing extra now, and it’s exactly what lets lab10 swap in a real database later without touching the controller.

Same pattern: new folder **Repositories**, then a new file **IContactRepository.cs** inside it:

```
using ContactManagement.Models;

namespace ContactManagement.Repositories;

public interface IContactRepository
{
    IEnumerable<Contact> GetAll();
    Contact? GetById(int id);
    Contact Create(Contact contact);
    bool Update(int id, Contact contact);
    bool Delete(int id);
}
```

Create **Repositories/InMemoryContactRepository.cs**:

```
using ContactManagement.Models;

namespace ContactManagement.Repositories;

public class InMemoryContactRepository : IContactRepository
{
    private readonly List<Contact> _contacts = new();
    private int _nextId = 1;

    public IEnumerable<Contact> GetAll() => _contacts;
}
```

```

public Contact? GetById(int id) => _contacts.FirstOrDefault(c => c.Id == id);

public Contact Create(Contact contact)
{
    contact.Id = _nextId++;
    _contacts.Add(contact);
    return contact;
}

public bool Update(int id, Contact updated)
{
    var existing = GetById(id);
    if (existing is null) return false;

    existing.Name = updated.Name;
    existing.Email = updated.Email;
    existing.Phone = updated.Phone;
    return true;
}

public bool Delete(int id)
{
    var existing = GetById(id);
    if (existing is null) return false;

    _contacts.Remove(existing);
    return true;
}
}

```

Step 4 — Wire it up with Dependency Injection

Open `Program.cs`. Right after `builder.Services.AddControllers()`, register the repository (see slide “Dependency Injection (DI)” — this uses `AddSingleton` so the same list survives across requests for the app’s lifetime):

```

using ContactManagement.Repositories;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddSingleton<IContactRepository, InMemoryContactRepository>();

var app = builder.Build();

app.MapControllers();

app.Run();

```

(Remove any leftover OpenAPI lines the template added — that’s lab12’s job, not lab6’s.)

Step 5 — The REST controller

This is the actual deliverable: “Define the REST endpoints for the CRUD operations, and implement them in controller-classes”. The `Controllers` folder already exists (from the template) — add `Controllers/ContactsController.cs` to it:

```
using ContactManagement.Models;
using ContactManagement.Repositories;
using Microsoft.AspNetCore.Mvc;

namespace ContactManagement.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ContactsController : ControllerBase
{
    private readonly IContactRepository _repository;

    public ContactsController(IContactRepository repository)
    {
        _repository = repository;
    }

    // GET api/contacts
    [HttpGet]
    public ActionResult<IEnumerable<Contact>> GetAll()
    {
        return Ok(_repository.GetAll());
    }

    // GET api/contacts/5
    [HttpGet("{id:int}")]
    public ActionResult<Contact> GetById(int id)
    {
        var contact = _repository.GetById(id);
        return contact is null ? NotFound() : Ok(contact);
    }

    // POST api/contacts
    [HttpPost]
    public ActionResult<Contact> Create([FromBody] Contact contact)
    {
        if (string.IsNullOrEmpty(contact.Name))
            return BadRequest("Name is required.");

        var created = _repository.Create(contact);
        return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
    }

    // PUT api/contacts/5
    [HttpPut("{id:int}")]
    public IActionResult Update(int id, [FromBody] Contact contact)
    {
        return _repository.Update(id, contact) ? NoContent() : NotFound();
    }
}
```

```

// DELETE api/contacts/5
[HttpDelete("{id:int}")]
public IActionResult Delete(int id)
{
    return _repository.Delete(id) ? NoContent() : NotFound();
}
}

```

Each method maps directly to the request-flow diagram from the slides: Routing → Controller/Action → Model Binding → Business Logic (here, the repository) → Response.

Step 6 — Run it and try every CRUD operation

Two ways to run it in VS Code:

- **Terminal:** `dotnet run` in the integrated terminal.
- **Debugger:** press F5. This uses the `.vscode/launch.json` generated in Step 1, runs the app, and stops on any breakpoints you set (click in the gutter left of a line number to add one — try the line inside `GetById`).

Either way, note the port it prints (e.g. `http://localhost:5236`).

Option A — `curl`, in a second terminal:

```

# Create
curl -X POST http://localhost:5236/api/contacts \
  -H "Content-Type: application/json" \
  -d '{"name":"Ada Lovelace","email":"ada@example.com","phone":"+43 1 234"}'

# Read all
curl http://localhost:5236/api/contacts

# Read one
curl http://localhost:5236/api/contacts/1

# Update
curl -X PUT http://localhost:5236/api/contacts/1 \
  -H "Content-Type: application/json" \
  -d '{"name":"Ada Byron","email":"ada@example.com","phone":"+43 1 234"}'

# Delete
curl -X DELETE http://localhost:5236/api/contacts/1

```

Option B — **REST Client extension:** create a file `requests.http` in the project root, paste the block below, adjust the port, then click the “Send Request” link that appears above each `###` block:

`@baseUrl = http://localhost:5236/api/contacts`

```

### Create
POST {{baseUrl}}
Content-Type: application/json

{
  "name": "Ada Lovelace",
  "email": "ada@example.com",

```

```

    "phone": "+43 1 234"
  }

### Read all
GET {{baseUrl}}

### Read one
GET {{baseUrl}}/1

### Update
PUT {{baseUrl}}/1
Content-Type: application/json

{
  "name": "Ada Byron",
  "email": "ada@example.com",
  "phone": "+43 1 234"
}

### Delete
DELETE {{baseUrl}}/1

```

Confirm: after POST you get a 201 with the new contact + an id; after DELETE, a follow-up GET /api/contacts/1 returns 404.

Checklist against the exercise

- ☒ Create a new ASP.Net WebApi Project — `dotnet new webapi`
- ☒ Define REST endpoints for CRUD, implemented in a controller class
- ☒ Model class for a Contact — `Models/Contact.cs`
- ☒ Collection of Contacts (as a list) — `InMemoryContactRepository`
- ☒ CRUD operations read from the HTTP request and store/query the collection